



Mait.me Progressive Staking

SECURITY ASSESSMENT REPORT

February 26, 2026

Prepared for:





Contents

1	About CODESPECT	2
2	Disclaimer	2
3	Risk Classification	3
4	Executive Summary	4
5	Audit Summary	5
5.1	Scope - Audited Files	5
5.2	Findings Overview	6
6	System Overview	7
6.1	Staking	7
6.2	Reward Calculation	7
6.3	Claiming Rewards	7
6.4	Withdrawal Flow	7
6.4.1	Withdrawal Cancellation	8
6.5	Emergency Mode	8
6.6	Treasury Management	8
6.7	Admin Stake Transfer	8
6.8	Tier Rate Updates	8
6.9	Access Control	9
6.10	Key Invariants	9
7	Issues	10
7.1	[Low] Withdraw execution leads to lost rewards if not enough assets held by treasury	10
7.2	[Low] Withdrawal request does not prevent rewards from accrual	10
7.3	[Info] Emergency withdrawals blocked in case of too many stakes	10
7.4	[Info] The MIN_STAKE_AMOUNT check is not applied during withdrawal requests	11
7.5	[Info] Withdrawals for web2 users may be delayed	11
7.6	[Best Practice] Withdrawal requests that are canceled or executed cannot be distinguished	12
8	Protocol Risks	13
9	Evaluation of Provided Documentation	14
10	Test Suite Evaluation	15
10.1	Compilation Output	15
10.2	Tests Output	15
10.3	Notes on the Test Suite	15



1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralized applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium** - Likely only under certain conditions or moderately incentivized.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilizes two other categories for findings: **Informational** and **Best Practices**.

- a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally.
- b) **Best Practices** findings indicate that certain portions of the code deviate from established smart contract development standards.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for the smart contract of Mait.me. Mait.me is an innovative web3 social platform.

This audit focuses on the ProgressiveStaking contract. The contract allows users to stake the MAIT token and collect rewards over time.

The audit was performed using:

- a) Manual analysis of the codebase.
- b) Dynamic analysis of smart contracts, execution testing.

CODESPECT found six points of attention, two classified as Low, three classified as Informational, and one classified as Best Practices. All of the issues are summarised in Table 2.

Organisation of the document is as follows:

- **Section 5** summarizes the audit.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the issues.
- **Section 8** presents the risks raised by the auditors.
- **Section 9** discusses the documentation provided by the client for this audit.
- **Section 10** presents the compilation and tests.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Low	0	2	0
Informational	0	3	0
Best Practices	0	1	0
Total	0	6	0

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	Mait.me Progressive Staking
Type of Project	Staking contract
Duration of Engagement	2 Days
Duration of Fix Review Phase	2 Days
Draft Report	January 28, 2026
Final Report	February 18 2026
Repository	maitime-contracts-staking
Commit (Audit)	ce61102843ceb1c27d875b394490ff859577016c
Commit (Final)	396f53ebae17add96ee702c749d044434fdd340a
Documentation Assessment	High
Test Suite Assessment	High
Auditors	shaflow01 , jecikpo

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	ProgressiveStaking.sol	404
	Total	404



5.2 Findings Overview

	Finding	Severity	Update
1	Withdraw execution leads to lost rewards if not enough assets held by treasury	Low	Fixed
2	Withdrawal request does not prevent rewards from accrual	Low	Fixed
3	Emergency withdrawals blocked in case of too many stakes	Info	Fixed
4	The MIN_STAKE_AMOUNT check is not applied during withdrawal requests	Info	Fixed
5	Withdrawals for web2 users may be delayed	Info	Fixed
6	Withdrawal requests that are canceled or executed cannot be distinguished	Best Practices	Fixed



6 System Overview

The MAIT.me ProgressiveStaking contract is a staking system that implements progressive interest rates based on staking duration. Users stake ERC20 tokens and earn rewards that increase across six tiers as their stake matures. The contract features a 90-day withdrawal notice period, automatic reward compounding across tier boundaries, and emergency withdrawal capabilities.

6.1 Staking

Users stake tokens via the `stake(...)` function. Each stake creates a new position with a unique `stakeId`, allowing users to have multiple concurrent positions with different start times. The position records the stake amount, start time, and last claim time for reward calculations.

```
function stake(uint256 amount) external nonReentrant whenNotPaused;
```

The function validates that the amount meets `MIN_STAKE_AMOUNT`, the contract is not paused, and emergency mode is not active.

6.2 Reward Calculation

Rewards are calculated based on time elapsed since the last claim, with compounding applied across tier boundaries. The `_calculatePositionRewards(...)` function iterates through each tier, calculating rewards for unclaimed time periods. Within a tier, simple interest is applied; across tiers, rewards compound into the principal for subsequent tier calculations.

```
// Per-tier reward formula
tierRewards = (currentAmount * tierRate * timeInTier)
              / (YEAR_DURATION * RATE_PRECISION);

// Cross-tier compounding
currentAmount += tierRewards;
```

Founder addresses (set at deployment) are excluded from rewards.

6.3 Claiming Rewards

Users can claim rewards for a single position or all positions at once. The `claimRewards(...)` function claims for a specific `stakeId`, while `claimAllRewards()` iterates through all user positions. Both functions update the `lastClaimTime` and transfer rewards from the treasury.

```
function claimRewards(uint256 stakeId) external nonReentrant whenNotPaused;

function claimAllRewards() external nonReentrant whenNotPaused;
```

Both functions revert if rewards are zero or if the treasury has insufficient balance to cover the payout.

6.4 Withdrawal Flow

Withdrawals follow a two-step process with a mandatory 90-day notice period. Users first request withdrawal via `requestWithdraw(...)`, which creates a `WithdrawRequest` struct storing the stake ID, amount, request time, and availability time. Partial withdrawals are supported.

```
function requestWithdraw(
    uint256 stakeId,
    uint256 amount
) external nonReentrant whenNotPaused;
```

Only one pending withdrawal request is allowed per `stakeId`, and users are limited to `MAX_PENDING_WITHDRAWALS` (10) total pending requests to prevent DoS via array growth.



After the 90-day notice period, users execute their withdrawal via `executeWithdraw(...)`. This function automatically claims any pending rewards before processing the withdrawal. If the position is fully withdrawn, it is removed from the user's stake array.

```
function executeWithdraw(uint256 stakeId) external nonReentrant;
```

Notably, `executeWithdraw(...)` does not have the `whenNotPaused` modifier, allowing users to withdraw even when the contract is paused. If the treasury has insufficient funds, rewards are silently skipped but principal withdrawal proceeds.

6.4.1 Withdrawal Cancellation

Users may cancel pending withdrawal requests at any time via `cancelWithdrawRequest(...)`. The stake remains active, and the user would need to submit a new request (restarting the 90-day timer) to withdraw later.

```
function cancelWithdrawRequest(uint256 stakeId) external nonReentrant;
```

6.5 Emergency Mode

The contract includes an irreversible emergency shutdown mechanism. When activated by an admin, `emergencyMode` is set to true and the contract is paused. Users can then call `emergencyWithdraw(...)` to immediately withdraw all stakes without waiting for the notice period.

```
function emergencyShutdown() external onlyRole(DEFAULT_ADMIN_ROLE);  
  
function emergencyWithdraw() external nonReentrant;
```

The `emergencyWithdraw(...)` function pays rewards on a best-effort basis, if the treasury has insufficient funds, only the principal is returned.

6.6 Treasury Management

The treasury holds tokens designated for reward payments, tracked separately from staked tokens via the `treasuryBalance` state variable. Admins manage the treasury through deposit and withdrawal functions.

```
function depositTreasury(uint256 amount)  
    external onlyRole(DEFAULT_ADMIN_ROLE);  
  
function withdrawTreasury(uint256 amount)  
    external onlyRole(DEFAULT_ADMIN_ROLE);
```

6.7 Admin Stake Transfer

Admins can transfer stake positions between users via `adminTransferStake(...)`. This feature is designed for web2-to-web3 user migration. The transfer preserves the original `startTime` and `lastClaimTime`, maintaining tier progression.

```
function adminTransferStake(  
    address fromUser,  
    uint256 stakeId,  
    address toUser  
) external nonReentrant onlyRole(ADMIN_ROLE);
```

Positions with pending withdrawal requests cannot be transferred.

6.8 Tier Rate Updates

Admins can update the APY rates for all tiers via `updateTierRates(...)`. Rate changes take effect immediately and apply retroactively to all unclaimed reward periods.

```
function updateTierRates(uint256[6] calldata newRates)  
    external onlyRole(DEFAULT_ADMIN_ROLE);
```

Rates are specified in basis points (e.g., 50 = 0.5%) and cannot exceed `RATE_PRECISION` (10000 = 100%).



6.9 Access Control

The contract uses OpenZeppelin's `AccessControl` with two roles:

- `DEFAULT_ADMIN_ROLE`: Full administrative access including treasury management, tier rate updates, emergency shut-down, and role management
- `ADMIN_ROLE`: Operational access for pause/unpause and stake transfers

6.10 Key Invariants

The contract maintains several critical invariants:

- $\text{totalStaked} + \text{treasuryBalance} \leq \text{actual token balance}$
- Each `stakeId` is globally unique (monotonically increasing counter)
- $\text{lastClaimTime} \geq \text{startTime}$ for all positions
- At most one pending withdrawal request per `stakeId`
- $\text{pendingWithdrawCount}[\text{user}] \leq \text{MAX_PENDING_WITHDRAWALS}$



7 Issues

7.1 [Low] Withdraw execution leads to lost rewards if not enough assets held by treasury

File(s): `ProgressiveStaking.sol`

Description: The `executeWithdraw(...)` function allows the holder of a stake to execute assets withdrawal. Inside the functions the rewards are calculated for the stake and are sent to the owner on condition that there is enough reward asset held at the contract:

```
if (rewards > 0 && treasuryBalance >= rewards) {
    treasuryBalance -= rewards;
    stakingToken.safeTransfer(msg.sender, rewards);
    emit RewardsClaimed(msg.sender, stakeId, rewards, block.timestamp);
}
```

While this is done according to the protocol's design, the issue is that if the withdrawal is partial and the `treasuryBalance` is insufficient the function still updates the `lastClaimTime`:

```
position.lastClaimTime = block.timestamp;
```

which means that the rewards for the remaining amount are also lost.

Impact: Loss of rewards for the remainder of the stake.

Recommendation(s): It is recommended to only update the `lastClaimTime` when the rewards are actually successfully claimed. This way in case the `treasuryBalance` is insufficient, the stake owner will still be able to collect rewards later for the remainder of the stake once the treasury is replenished.

Status: Fixed

7.2 [Low] Withdrawal request does not prevent rewards from accrual

File(s): `ProgressiveStaking.sol`

Description: The `ProgressiveStaking` contracts provides a two-step withdrawal process where the staker first needs to call `requestWithdraw(...)`, then `executeWithdraw(...)`. where the `executeWithdraw(...)` can be called after `NOTICE_PERIOD` passes. The problem is that there is no effect on the stake after calling `requestWithdraw(...)`, so a user could call it immediately after creating a stake and then wait just until he wants to unstake and call `executeWithdraw(...)`, which in practical terms mean that the `NOTICE_PERIOD` can be ignored as long as the user calls `requestWithdraw(...)` early.

Impact: Practical lack of expected notice period for withdrawing the stake.

Recommendation(s): The `requestWithdraw(...)` function should stop accrual of the rewards.

Status: Fixed

7.3 [Info] Emergency withdrawals blocked in case of too many stakes

File(s): `ProgressiveStaking.sol`

Description: For the `StakePosition` dynamic array, there is no maximum length limit. Although the array decreases when positions are removed, a user holding too many positions at the same time could have the emergency withdrawals affected. Positions of `Web2` accounts, are managed under a single admin address. If this address holds an excessive number of users' positions, in case of a security issue, emergency withdrawals may fail due to out-of-gas errors. And it may affect the `calculateTotalRewards(...)` view function. The emergency withdrawal functionality will reach its gas limits between 7,672 and 8,172 amount of positions for a single address.

Impact: Emergency withdrawal functionality not available for `Web2` accounts.

Recommendation(s): Place a reasonable limit on how many stakes each address can open. For `Web2` distribute the stakes across multiple custodial addresses.

Status: Fixed

7.4 [Info] The MIN_STAKE_AMOUNT check is not applied during withdrawal requests

File(s): `ProgressiveStaking.sol`

Description: In the `stake(...)` function, there is a position size check that requires the stake amount to be greater than `MIN_STAKE_AMOUNT`.

```
function stake(uint256 amount) external nonReentrant whenNotPaused {
    if (amount == 0) revert ZeroAmount();
    if (amount < MIN_STAKE_AMOUNT) revert StakeAmountTooLow();
    //...
}
```

However, in the `requestWithdraw(...)` function, there is no check to ensure that the remaining position size after withdrawal is still greater than `MIN_STAKE_AMOUNT`. This may result in positions smaller than `MIN_STAKE_AMOUNT` existing in the contract.

Impact: Unexpected small positions may appear in the contract, which is unfavorable for management.

Recommendation(s): It is recommended to add a check in `requestWithdraw(...)`. If the remaining position size is not 0, it must be greater than `MIN_STAKE_AMOUNT`.

Status: Fixed

7.5 [Info] Withdrawals for web2 users may be delayed

File(s): `ProgressiveStaking.sol`

Description: The document mentions that for web2 users in the system, tokens are managed in a single admin account. However, a single account can have at most 10 pending withdrawal requests within 90 days.

```
function requestWithdraw(uint256 stakeId, uint256 amount) external nonReentrant whenNotPaused {
    //...
    if (pendingWithdrawCount[msg.sender] >= MAX_PENDING_WITHDRAWALS) revert TooManyPendingWithdrawals();
    //...
}
```

This makes it difficult for a single account to manage too many users, as their withdrawal requests may be delayed due to the admin account reaching the maximum number of pending requests.

Impact: Designing the system to manage all web2 users' tokens in a single admin account may be unfeasible.

Recommendation(s): It is recommended to delegate web2 users in batches to multiple accounts or to implement a dedicated withdrawal function for this special case.

Status: Fixed



7.6 [Best Practice] Withdrawal requests that are canceled or executed cannot be distinguished

File(s): `ProgressiveStaking.sol`

Description: Calling the `executeWithdraw(...)` function to process a withdrawal request sets the `executed` field to `true`.

```
function executeWithdraw(uint256 stakeId) external nonReentrant {  
    //...  
    request.executed = true;  
    //...  
}
```

Calling the `cancelWithdrawRequest(...)` function to cancel a withdrawal request also sets the `executed` field to `true`.

```
function cancelWithdrawRequest(uint256 stakeId) external nonReentrant {  
    //...  
    request.executed = true; // Mark as executed to prevent reuse  
    //...  
}
```

There is no other field to distinguish canceled and executed withdrawal requests in the history.

Impact: The historical withdrawal records returned by the `getPendingWithdrawals(...)` function cannot distinguish between canceled and executed requests.

Recommendation(s): It is recommended to add a field to distinguish between canceled and executed withdrawal requests.

Status: Fixed



8 Protocol Risks

This section outlines protocol-level risks as identified and understood by the audit team. These points reflect design choices, assumptions, or trust dependencies observed during the review and are documented to provide transparency regarding the protocol's risk profile.

- **Centralisation Risk - Admin Stake Transfer:** The `adminTransferStake(...)` function grants addresses holding the `ADMIN_ROLE` the capability to unilaterally transfer any user's stake position to an arbitrary recipient address. This transfer occurs without requiring the stake owner's consent or signature, and preserves the original `startTime` and `lastClaimTime`, meaning the recipient inherits the full tier progression and any unclaimed rewards. While the function is intended for legitimate use cases such as web2-to-web3 user migration, it effectively allows privileged administrators to seize user funds at will. Users must trust that MAIT.me securely manages the private keys associated with the `ADMIN_ROLE` and maintains robust operational controls to prevent unauthorized or malicious stake transfers.
- **Centralisation Risk - Treasury Management:** The `DEFAULT_ADMIN_ROLE` has unrestricted control over the reward treasury via `depositTreasury(...)` and `withdrawTreasury(...)` functions. An admin may withdraw the entire treasury balance at any time without restriction, which would prevent users from claiming their earned rewards. Additionally, the `updateTierRates(...)` function allows admins to retroactively modify APY rates, affecting all unclaimed reward periods. Setting rates to zero would effectively eliminate pending rewards for all stakers. Users must trust that administrators will not abuse these capabilities to the detriment of stakers.
- **Reward Claiming - Tier System Design:** Compounding occurs when a reward calculation spans multiple tiers within a single call. Rewards earned in tier N are added to the principal before calculating tier N+1 rewards. This creates a cascading effect where earlier tier earnings contribute to the base for subsequent tier calculations. The practical implication of such design is that users who claim rewards less frequently receive slightly higher total rewards than users who claim at each tier boundary. It is recommended to document this clearly.



9 Evaluation of Provided Documentation

The **Mait.me** documentation was provided primarily through comprehensive NatSpec comments embedded directly within the codebase, the README and ARCHITECTURE files:

- **NatSpec:** The in-code NatSpec comments were clear, detailed, and highly effective in explaining specific flows, design intentions, and conditional branches. Each core functionality was well documented, allowing for an efficient understanding of the system's behavior and expected invariants.
- **Markdown files:** The README and ARCHITECTURE files provided good high-level overview of the contract it's functionality and main design principles.

Overall, the documentation was adequate and fully sufficient for the scope of this audit. Moreover, the Mait.me team remained consistently available and responsive, promptly addressing all questions and clarifications raised by **CODESPECT**, which significantly streamlined the audit process.



10 Test Suite Evaluation

10.1 Compilation Output

```
> forge build
Receiving objects: 100% (47/47), 24.40 KiB | 304.00 KiB/s, done.
Resolving deltas: 100% (19/19), done.
[] Compiling...
[] Compiling 49 files with Solc 0.8.24
[] Solc 0.8.24 finished in 3.49s
Compiler run successful with warnings:
[...]
```

The warnings are related to tests and hence were omitted in the output.

10.2 Tests Output

```
> forge test
[] Compiling...
No files changed, compilation skipped
Ran 20 tests for test/ProgressiveStaking.t.sol:ProgressiveStakingTest
[...]
```

Test Suite	Passed	Failed	Skipped	Finished In	CPU Time
test/ProgressiveStaking.t.sol:ProgressiveStakingTest	20	0	0	11.32ms	2.73ms
test/ProgressiveStaking.time.t.sol:ProgressiveStakingTimeTest	12	0	0	11.39ms	2.77ms
test/ProgressiveStaking.events.t.sol:ProgressiveStakingEventsTest	13	0	0	11.25ms	1.51ms
test/ProgressiveStaking.validation.t.sol:ProgressiveStakingValidationTest	43	0	0	29.91s	5.08ms
test/ProgressiveStaking.fuzz.t.sol:ProgressiveStakingFuzzTest	7	0	0	30.56s	179.51ms
test/ProgressiveStaking.dos.t.sol:ProgressiveStakingDoSTest	10	0	0	30.56s	8.00ms
test/ProgressiveStaking.access.t.sol:ProgressiveStakingAccessTest	33	0	0	30.81s	4.35ms
test/ProgressiveStaking.invariant.t.sol:ProgressiveStakingInvariantTest	10	0	0	30.81s	219.64s
Total	148	0	0	30.82s	152.69s

```
[...]
```

10.3 Notes on the Test Suite

The Mait.me test suite provides strong coverage of the core contract components within the audit scope. It explores a broad range of functional flows and verifies key invariants that must be preserved. Additionally, the suite includes several edge cases that are expected to be handled by explicit conditions within the contract logic. It contains also extensive privilege checks, DoS scenarios and tests gas limitations.